14. Alpha-Beta Pruning Algorithm

```
def alpha_beta_pruning(depth, node_idx, is_max, values, alpha, beta):
    # Base case: Leaf node level reached at depth 3
    if depth == 3:
        return values[node_idx]

    if is_max:
        best = float('-inf')
        for i in range(2):
            val = alpha_beta_pruning(depth + 1, node_idx * 2 + i, False, values, alpha, beta)
            best = max(best, val)
            alpha = max(alpha, best)
            if beta <= alpha:
                print(f"  Pruning evaluation branch at Max node index {node_idx}")
                break
        return best
    else:
        best = float('inf')
        for i in range(2):
            val = alpha_beta_pruning(depth + 1, node_idx * 2 + i, True, values, alpha, beta)
            best = min(best, val)
            beta = min(beta, best)
            if beta <= alpha:
                print(f"  Pruning evaluation branch at Min node index {node_idx}")
                break
        return best

if __name__ == "__main__":
    leaf_values = [3, 5, 6, 9, 1, 2, 0, -1]
    print("Beginning alpha-beta depth traversal:")
    result = alpha_beta_pruning(0, 0, True, leaf_values, float('-inf'), float('inf'))
```

print(f"\nOptimal value found with pruning: {result}")

